



Smart Contract Fuzzing Towards Profitable Vulnerabilities

Ziqiao Kong, Cen Zhang, Maoyi Xie, Ming Hu, Yue Xue, Ye Liu, Haijun Wang, Yang Liu

FSE'25

Motivation



On-Chain Vulnerabilities



On-Chain Vulnerabilities



BlockSec Phalcon 

@Phalcon_xyz



ALERT! Our system has detected a suspicious transaction targeting an unknown project on [#Base](#), resulting in a loss of approximately \$1M. The

On-Chain Vulnerabilities



BlockSec Phalcon ✓

@Phalcon_xyz



ALEF
unkr



PeckShield Inc. ✓

@peckshield



It seems today's victim [@OnyxDAO](#) (w/ >\$3.8m loss) falls prey to a known precision issue in forked CompoundV2 code base. The drained funds include 4.1m VUSD, 7.35m XCN, 5k DAI, 0.23 WBTC, 50k USDT.

On-Chain Vulnerabilities



BlockSec Phalcon ✓

@Phalcon_xyz

ALEF
unkr



PeckShield Inc. ✓

@peckshield

It seems today's victim [@OnyxDAO](#) (known precision issue in forked Cor funds include 4.1m VUSD, 7.35m XCM



CertiK Alert ⚙️

@CertiKAlert

#CertiKInsight 🚨

On 26 September 2024, [@Bedrock_DeFi](#) was exploited for ~\$1.7M
etherscan.io/tx/0x725f0d653...

The mint function didn't check that msg.value was native BTC which allowed the attacker to swap 30.8 ETH 1 to 1 for uniBTC.

```
/**
 * @dev mint uniBTC with native BTC
 */
function mint() external payable {
    require(!paused[NATIVE_BTC], "SYS002");
    _mint(msg.sender, msg.value);
}
```

On-Chain Vulnerabilities



BlockSec Phalcon ✓

@Phalcon_xyz

ALEF
unkr



PeckShield Inc. ✓

@peckshield

It seems today's victim [@OnyxDAO](#) (known precision issue in forked Cor funds include 4.1m VUSD, 7.35m XCM

Exploits happen every day!



CertiK Alert ⚙️

@CertiKAlert

#CertiKInsight 🚨

On 26 September 2024, [@Bedrock_DeFi](#) was exploited for ~\$1.7M
etherscan.io/tx/0x725f0d653...

The mint function didn't check that msg.value was native BTC which allowed the attacker to swap 30.8 ETH 1 to 1 for uniBTC.

```
/**
 * @dev mint uniBTC with native BTC
 */
function mint() external payable {
    require(!paused[NATIVE_BTC], "SYS002");
    _mint(msg.sender, msg.value);
}
```



Research Gap

- Profitable bugs draw more attention for on-chain contracts
 - Economics nature of smart contracts and high volume of web3 trading
 - Deployed contracts are very hard to patch, without a proxy pattern
- Proof-of-Concept (PoC) is far not enough
 - *Huge gap* between “discovery” and “exploitation”
 - “Dark Forest” nature^[1]: Only full exploitation/rescue makes sense in the arm race
- Proof-of-Profit (PoP) is crucial
 - Fast synthesis and maximized profits
 - Limited research

[1] <https://www.paradigm.xyz/2020/08/ethereum-is-a-dark-forest>



Proof-of-Profit Hunting

- Solve stateful constraints
 - Liquidity, buy/sell locks
 - Blocks, timestamps etc
- Effective search strategy
 - More fuzzing power on seeds making more profits
- Careful oracle design
 - Multi-token accounting
 - False positive = failure in on-chain arm race = lose money (even others'^[1])

[1] <https://www.paradigm.xyz/2020/08/ethereum-is-a-dark-forest>



Previous Fuzzers

- Echidna [ISSTA'20]
 - No oracle
- ConFuzzius [Euro S&P'20], Smartian [ASE'21]
 - No multi contract/on-chain fuzzing support
- EF/CF [Euro S&P'23]
 - Focus on profitable vulnerabilities but limited oracle
 - Incomplete multi contract and on-chain fuzzing support
- ItyFuzz [ICSE'23]
 - High throughput, decent design
 - High false positive rate



Previous Fuzzers

- Echidna [ISSTA'20]
 - No oracle
- ConFuzzius [Euro S&P'20], Smartian [ASE'21]
 - No multi contract/on-chain fuzzing support
- EF/CF [Euro S&P'23]
 - Focus on profitable vulnerabilities but limited oracle
 - Incomplete multi contract and on-chain fuzzing support
- ItyFuzz [ICSE'23]
 - High throughput, decent design
 - High false positive rate

VERITE: Fast profitable vulnerabilities hunting without false positive

Approach



Overview

- Actions based mutation
- Profit accounting
- Profit potential identification
- Stochastic Gradient Descent Based Exploitation



Action Based Mutation



Action Based Mutation

```
1 function swap2_action(address token_from, address token_to, uint percentage) {  
2     uint amount = token_from.balanceOf(address(this)) * percentage / 100;  
3  
4     token_from.approve(Router, amount);  
5     address[] memory path = new address[](2);  
6     path[0] = token_from;  
7     path[1] = token_to;  
8     Router.swapExactETHForTokensSupportingFeeOnTransferTokens(amount, path,  
9     ...);  
9 }
```



Action Based Mutation

“Swap {percentage}% tokens from {token_from} to {token_to}”

```
1 function swap2_action(address token_from, address token_to, uint percentage) {  
2     uint amount = token_from.balanceOf(address(this)) * percentage / 100;  
3  
4     token_from.approve(Router, amount);  
5     address[] memory path = new address[](2);  
6     path[0] = token_from;  
7     path[1] = token_to;  
8     Router.swapExactETHForTokensSupportingFeeOnTransferTokens(amount, path,  
9     ...);  
9 }
```




Action Based Mutation

“Swap {percentage}% tokens from {token_from} to {token_to}”

```
1 function swap2_action(address token_from, address token_to, uint percentage) {  
2     uint amount = token_from.balanceOf(address(this)) * percentage / 100;  
3  
4     token_from.approve(Router, amount);  
5     address[] memory path = new address[](2);  
6     path[0] = token_from;  
7     path[1] = token_to;  
8     Router.swapExactETHForTokensSupportingFeeOnTransferTokens(amount, path,  
9     ...);  
9 }
```



Action Based Mutation

“Swap {percentage}% tokens from {token_from} to {token_to}”

```
1 function swap2_action(address token_from, address token_to, uint percentage) {  
2     uint amount = token_from.balanceOf(address(this)) * percentage / 100;  
3  
4     token_from.approve(Router, amount);  
5     address[] memory path = new address[](2);  
6     path[0] = token_from;  
7     path[1] = token_to;  
8     Router.swapExactETHForTokensSupportingFeeOnTransferTokens(amount, path,  
9     ...);  
}
```

Implement action by calling corresponding contracts



Action Based Mutation

“Swap {percentage}% tokens from {token_from} to {token_to}”

```
1 function swap2_action(address token_from, address token_to, uint percentage) {  
2     uint amount = token_from.balanceOf(address(this)) * percentage / 100;  
3  
4     token_from.approve(Router, amount);  
5     address[] memory path = new address[](2);  
6     path[0] = token_from;  
7     path[1] = token_to;  
8     Router.swapExactETHForTokensSupportingFeeOnTransferTokens(amount, path,  
9     ...);  
9 }
```



Action Based Mutation

“Swap {percentage}% tokens from {token_from} to {token_to}”

```
1 function swap2_action(address token_from, address token_to, uint percentage) {  
2     uint amount = token_from.balanceOf(address(this)) * percentage / 100;  
3  
4     token_from.approve(Router, amount);  
5     address[] memory path = new address[](2);  
6     path[0] = token_from; Satisfy inter-constraints of multiple contracts  
7     path[1] = token_to;  
8     Router.swapExactETHForTokensSupportingFeeOnTransferTokens(amount, path,  
9     ...);  
9 }
```

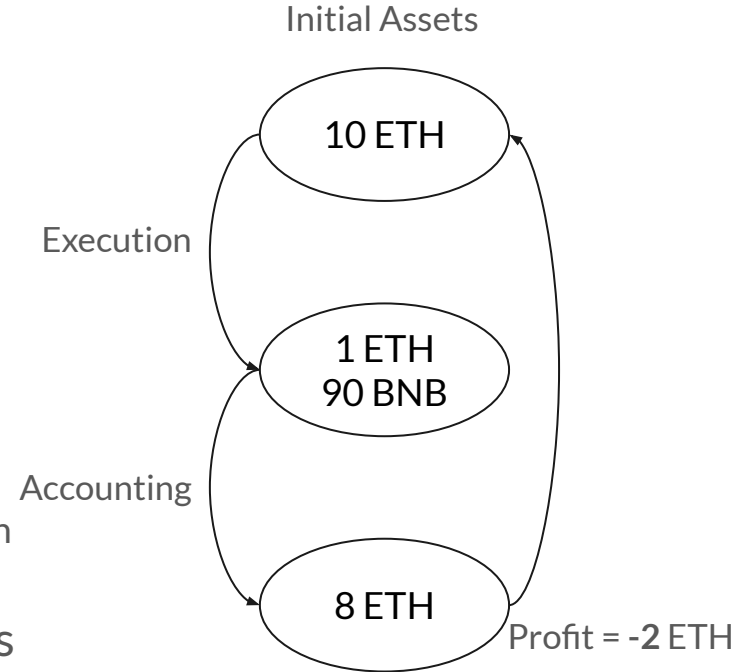


Action Based Mutation

- **Actions:** Abstractions of high-level DeFi actions
 - Avoid meaningless mutation
- VERITE extracts actions related to tokens and DEXs
 - We provide an LLM approach to semi-automatically generate actions
- A single transaction is a “fallback” action
 - For general applicability
 - Scheduling priority is given to defined actions

Profit Accounting

- **Profit:** Delta of assets in one pricing token
 - Common to have multiple assets during testing
 - Accounting multiple assets via DEX actions
 - Profit could be negative
- Always accounting on-chain
 - Price is never fixed
 - Locally verified by Ethereum node implementation
 - Eliminate false positives
- **Proof-of-Profits:** Seeds with positive profits





Profit Potential Identification

- **Profit Potential:** The possibility of a seed making profits
- Seeds that abnormally affects price
 - Likely to cause price manipulation
- **“Unconditional Tokens Gains”**
 - Likely to cause inflation and manipulate price
 - Might not really make profit due to lack of liquidity or so
- **“Unconditional Tokens Burns”**
 - Likely to cause deflation and manipulate price

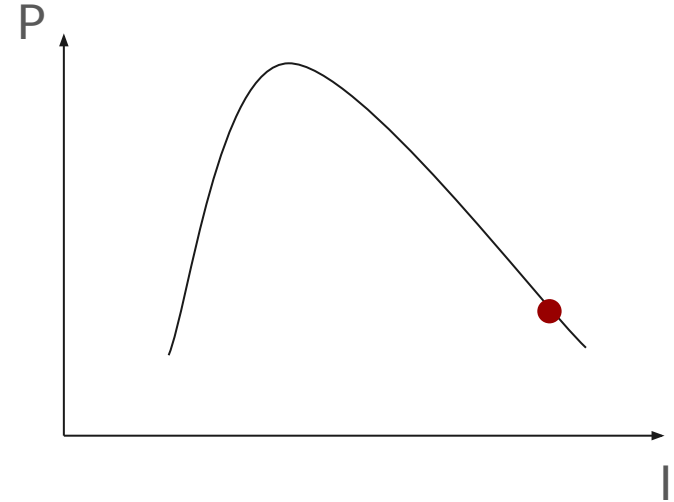
Stochastic Gradient Descent Based Exploitation

- **Profit function: $P = F(I)$**
 - P is calculated by profit accounting
 - I includes all arguments of the transactions
 - Only mutate arguments



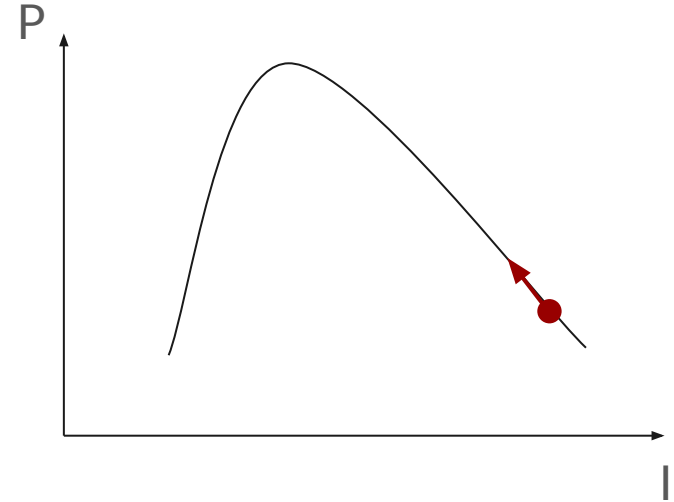
Stochastic Gradient Descent Based Exploitation

- **Profit function:** $P = F(I)$
 - P is calculated by profit accounting
 - I includes all arguments of the transactions
 - Only mutate arguments



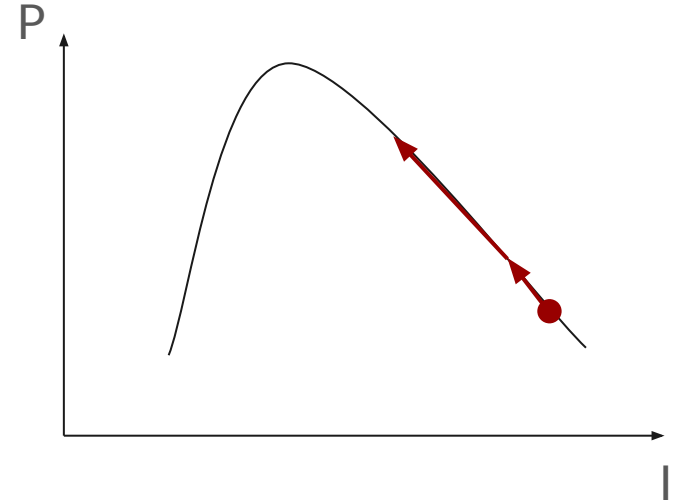
Stochastic Gradient Descent Based Exploitation

- **Profit function:** $P = F(I)$
 - P is calculated by profit accounting
 - I includes all arguments of the transactions
 - Only mutate arguments



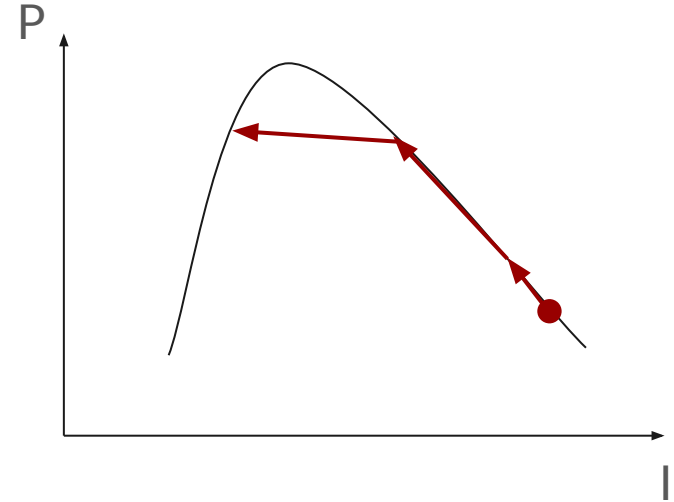
Stochastic Gradient Descent Based Exploitation

- **Profit function:** $P = F(I)$
 - P is calculated by profit accounting
 - I includes all arguments of the transactions
 - Only mutate arguments



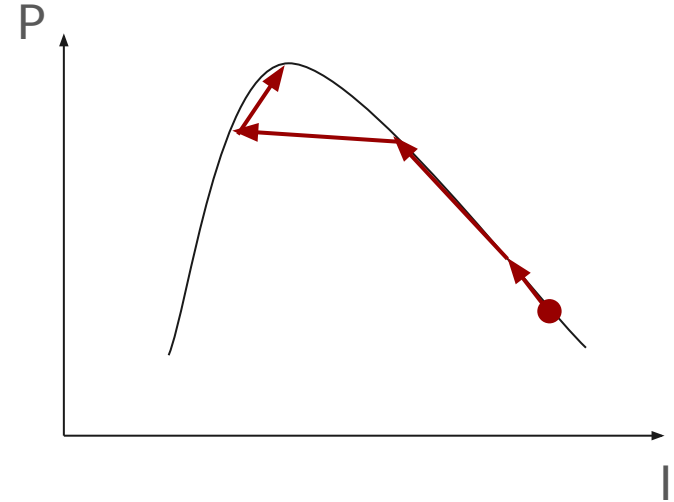
Stochastic Gradient Descent Based Exploitation

- **Profit function:** $P = F(I)$
 - P is calculated by profit accounting
 - I includes all arguments of the transactions
 - Only mutate arguments



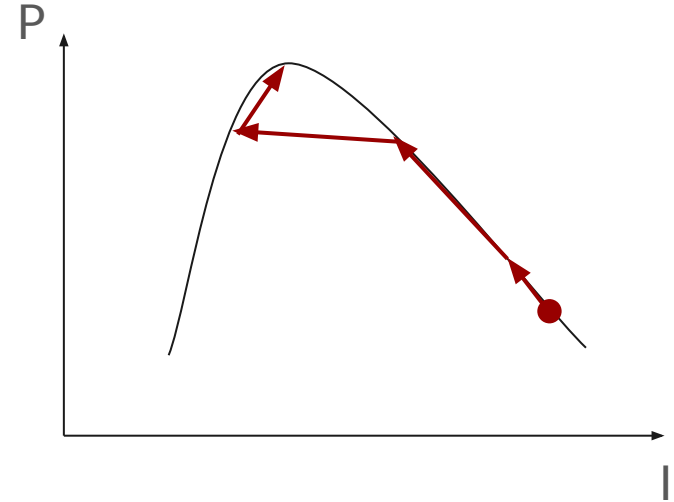
Stochastic Gradient Descent Based Exploitation

- **Profit function:** $P = F(I)$
 - P is calculated by profit accounting
 - I includes all arguments of the transactions
 - Only mutate arguments



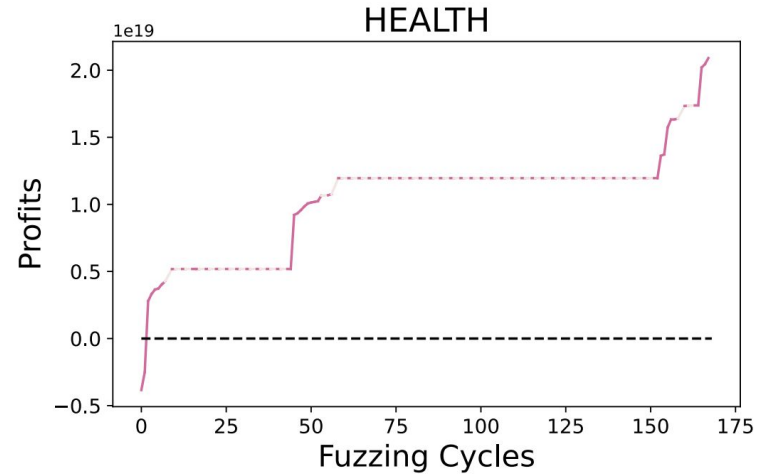
Stochastic Gradient Descent Based Exploitation

- **Profit function:** $P = F(I)$
 - P is calculated by profit accounting
 - I includes all arguments of the transactions
 - Only mutate arguments
- Only seeds with profit potential go through exploitation
 - SGD is slow and very expensive



Stochastic Gradient Descent Based Exploitation

- **Profit function: $P = F(I)$**
 - P is calculated by profit accounting
 - I includes all arguments of the transactions
 - Only mutate arguments
- Only seeds with profit potential go through exploitation
 - SGD is slow and very expensive
- Negative profits could go positive!



Evaluation



Experiment

- Implementation from scratch based on LibAFL and revm
- Dataset extended from ItyFuzz, derived from DeFiHackLabs
 - 61 incidents, averagely 1.1M loss
 - Wide coverage of bug types
- Baseline: ItyFuzz and *Real-World attacking transactions*
 - Evaluate if VERITE can outperform in the arm race
- Other baselines like ConFuzzious, EF/CF find 0 vulnerability
 - Limited support for multiple contracts
 - No profit accounting



RQ1: End-to-End Comparison

- 29 targets exploited with 18 millions profits extracted
- Outperforms ItyFuzz on all targets
 - With even higher coverage & throughput
- No false positive
- **Extract more profits than real-world transactions on 12 incidents**
 - 11.45 times more in the best case
 - Highlight the superior of VERITE in the on-chain arm race



RQ1: Case Study of *Bego*

```
function _mint(address account, uint256 amount) internal {  
    require(account != address(0), "BEP20: mint to the zero address");  
    // ...  
    _balances[account] = _balances[account].add(amount);  
    // ...  
}
```



RQ1: Case Study of *Bego*

```
function _mint(address account, uint256 amount) internal {  
    require(account != address(0), "BEP20: mint to the zero address");  
    // ...  
    _balances[account] = _balances[account].add(amount);  
    // ...  
}
```

Insufficient checks



RQ1: Case Study of *Bego*

```
function _mint(address account, uint256 amount) internal {  
    require(account != address(0), "BEP20: mint to the zero address");  
    // ...  
    _balances[account] = _balances[account].add(amount);  
    // ...  
}
```

Insufficient checks

Trivial “Unconditional Tokens Gain” vulnerability.

RQ1: Case Study of *Bego*

 BSC

0x9f4ef3cc55b016ea6b867807a09f80d1b2e36f6cd6fccfaf0182f46060332c57  

Info

Execution Trace

Token Flow

Call Graph

Balance Changes

Storage Changes

Nonce Cha

▼

Execution Trace

 Filter by contract label/address, function name

☐

 Jump

☐

 Static Call

☐

0

▼

[Sender]

0xde01f6Ce91E4F4bdB94BB934d30647d72182320F

1

▼



0 → CALL

[Receiver]

0x08a525104Ea2A92aBbcE8e4e61C667eED56f3B42

call

()

calldata

▶

()

2

▼



1 → CALL

Binance GeoDB Coin

mint

(_amount = 1,000,000,000,000,000,000,000,000,000,000 ,

<https://skyflow.certik.com/tx/bsc/0x9f4ef3cc55b016ea6b867807a09f80d1b2e36f6cd6fccfaf0182f46060332c57>

RQ1: Case Study of *Bego*

BSC

0x9f4ef3cc55b016ea6b867807a09f80d1b2e36f6cd6fccfaf0182f46060332c57 📄 📊

Info Execution Trace Token Flow Call Graph Balance Changes Storage Changes Nonce Changes

Execution Trace

Filter by contract label/address, function name 🔍

☐ Jump ☐ Static Call ☐

0	▼	[Sender]	0xde01f6Ce91E4F4bdB94BB934d30647d72182320F
1	▼	🌀 0 → CALL [Receiver] 0x08a525104Ea2A92aBbcE8e4e61C667eED56f3B42	call calldata
2	▼	🌀 1 → CALL Binance GeoDB Coin . mint	(_amount = 1,000,000,000,000,000,000,000,000,000,000)

Obvious manual value

<https://skyflow.certik.com/tx/bsc/0x9f4ef3cc55b016ea6b867807a09f80d1b2e36f6cd6fccfaf0182f46060332c57>

RQ1: Case Study of *Bego*

BSC

0x9f4ef3cc55b016ea6b867807a09f80d1b2e36f6cd6fccfaf0182f46060332c57

Info

Execution Trace

Token Flow

Call Graph

Balance Changes

Storage Changes

Nonce Chg

Execution Trace

Filter by contract label/address, function name

Jump

Static Call

0

[Sender]

0xde01f6Ce91E4F4bdB94BB934d30647d72182320F

1

CALL

[Receiver]

0x08c525404Fc2A92aBbcF8e4e61C663eFD56f3B42

2

CALL

Binance GeoDB Coin .

mint

(_amount = 1,000,000,000,000,000,000,000,000,000)

<https://skyflow.certik.com/tx/bsc/0x9f4ef3cc55b016ea6b867807a09f80d1b2e36f6cd6fccfaf0182f46060332c57>



RQ1: Case Study of *sut*

```
function buyTokens(uint256 _numberOfTokens) public payable {  
    uint256 _bnbvalue = (_numberOfTokens/1000000000000000000)*tokenPrice;  
    require(msg.value >= _bnbvalue);  
    // ...  
    require(  
        tokenContract.transfer(msg.sender, _numberOfTokens), "Transfer failed");  
    // ...  
}
```

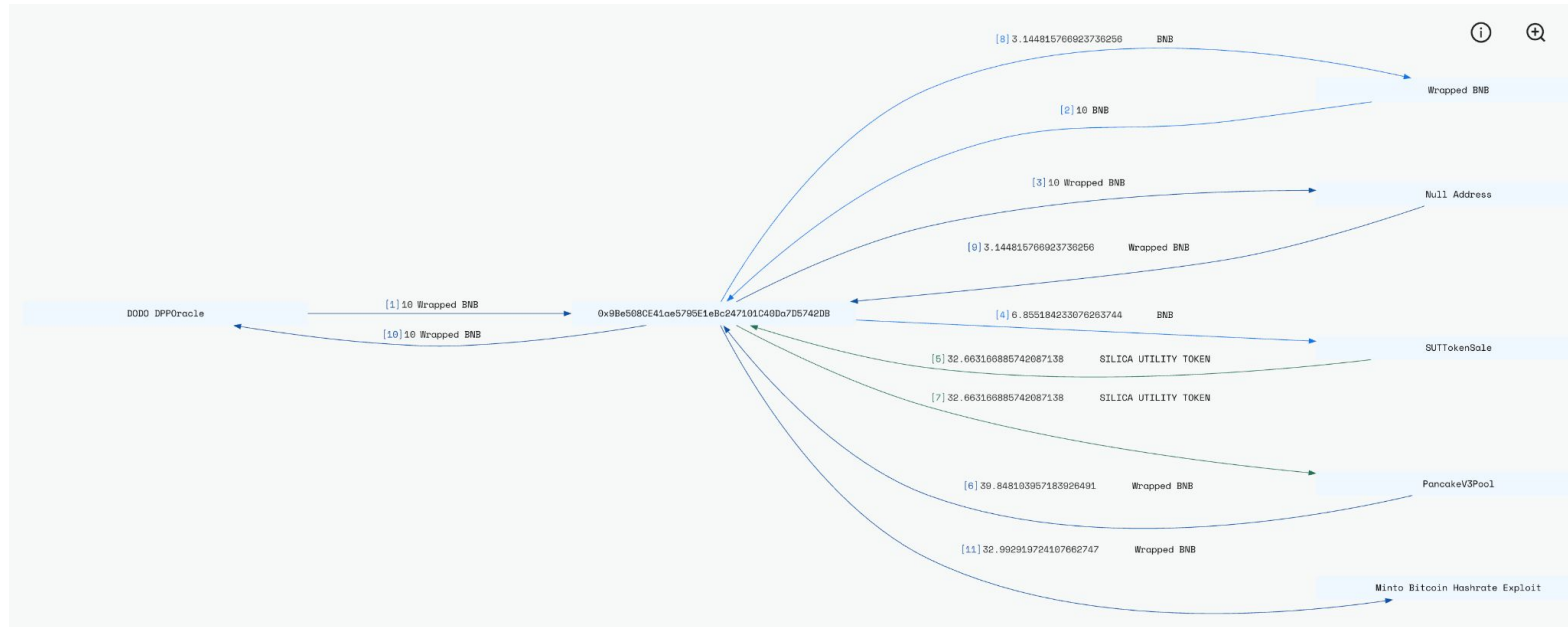


RQ1: Case Study of *sut*

```
function buyTokens(uint256 _numberOfTokens) public payable {  
    uint256 _bnbvalue = (_numberOfTokens/1000000000000000000)*tokenPrice;  
    require(msg.value >= _bnbvalue);  
    // ...  
    require(  
        tokenContract.transfer(msg.sender, _numberOfTokens), "Transfer failed");  
    // ...  
}
```

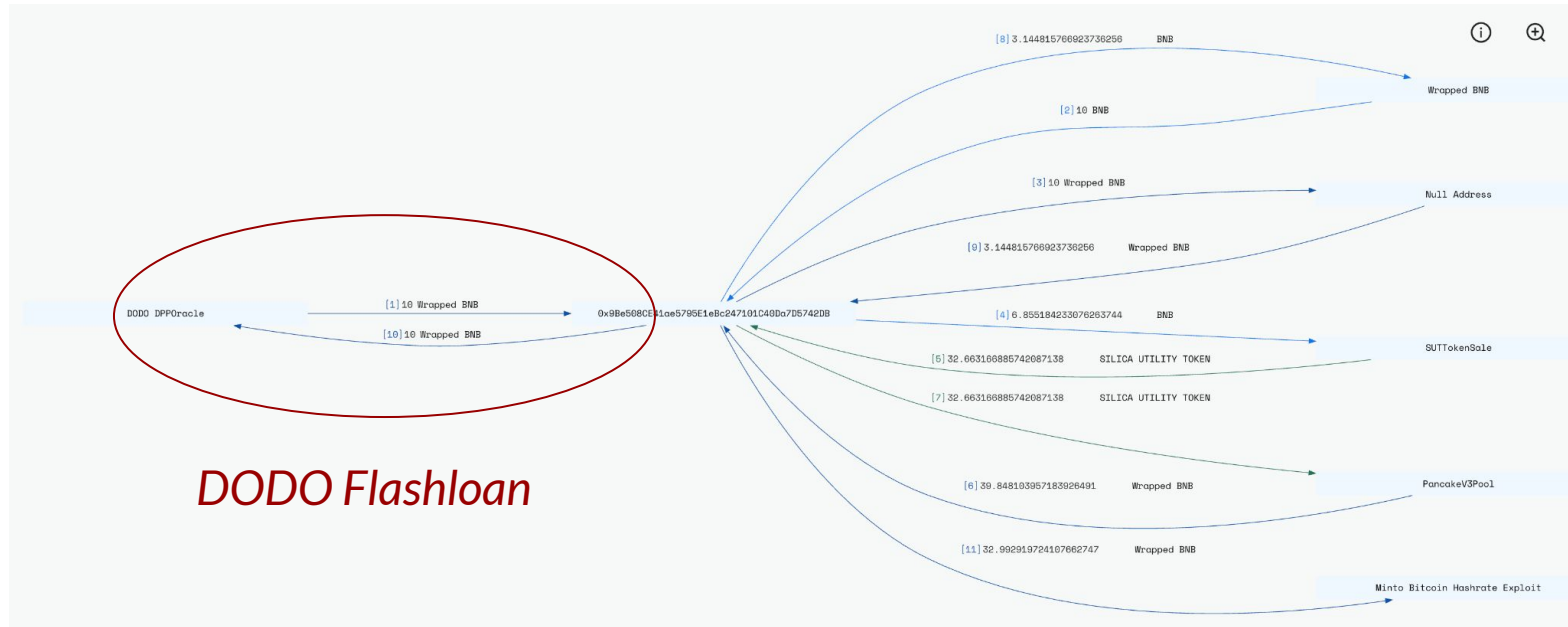
Fixed price, vulnerable to price manipulation

RQ1: Case Study of *sut*

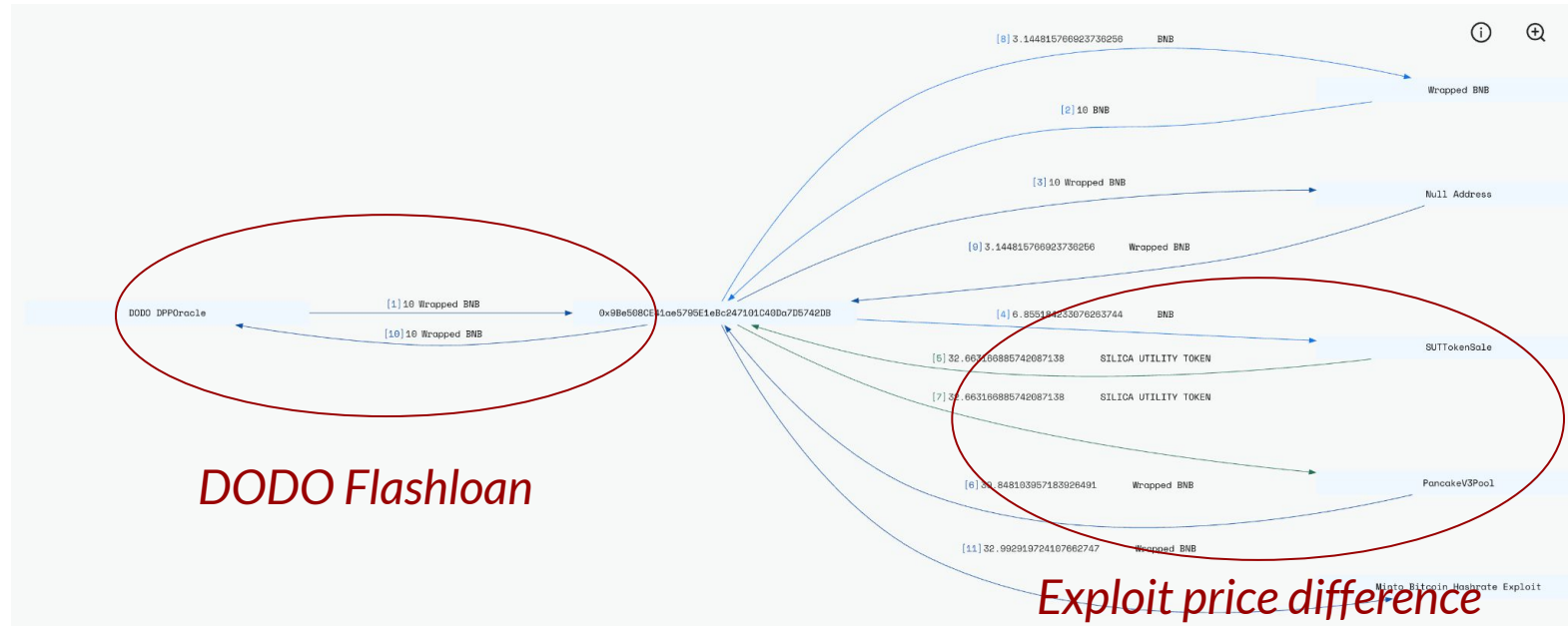


<https://skyflow.certik.com/tx/bsc/0xfa1ece5381b9e2b2b83cb10faefde7632ca411bb38dd6bafef1f1140b1360f6ae>

RQ1: Case Study of *sut*



RQ1: Case Study of *sut*





RQ1: Case Study of *sut*

```
function buyTokens(uint256 _numberOfTokens) public payable {  
    uint256 _bnbvalue = (_numberOfTokens/1000000000000000000)*tokenPrice;  
    require(msg.value >= _bnbvalue);  
    // ...  
    require(  
        tokenContract.transfer(msg.sender, _numberOfTokens), "Transfer failed");  
    // ...  
}
```

How VERITE exploits this better?

Fixed price, vulnerable to price manipulation



RQ1: Case Study of *sut*

```
function buyTokens(uint256 _numberOfTokens) public payable {  
    uint256 _bnbvalue = (_numberOfTokens/1000000000000000000)*tokenPrice;  
    require(msg.value >= _bnbvalue); Underflow to 0 if {_numberOfTokens} too small  
    // ...  
    require(  
        tokenContract.transfer(msg.sender, _numberOfTokens), "Transfer failed");  
    // ...  
}
```

Fixed price, vulnerable to price manipulation



RQ1: Case Study of *sut*

```
function buyTokens(uint256 _numberOfTokens) public payable {  
    uint256 _bnbvalue = (_numberOfTokens/1000000000000000000)*tokenPrice;  
    require(msg.value >= _bnbvalue); Underflow to 0 if {_numberOfTokens} too small  
    // ... Pass this  
    require(  
        tokenContract.transfer(msg.sender, _numberOfTokens), "Transfer failed");  
    // ...  
}
```

Fixed price, vulnerable to price manipulation

RQ1: Case Study of *sut*

```
function buyTokens(uint256 _numberOfTokens) public payable {  
    uint256 _bnbvalue = (_numberOfTokens/1000000000000000000)*tokenPrice;  
    require(msg.value >= _bnbvalue);  
    // ...  
    require(  
        tokenContract.transfer(msg.sender, _numberOfTokens), "Transfer failed");  
    // ...  
}
```

Underflow to 0 if {_numberOfTokens} too small

Pass this

Transfer arbitrary tokens

Fixed price, vulnerable to price manipulation

RQ1: Case Study of *sut*

```
function buyTokens(uint256 _numberOfTokens) public payable {  
    uint256 _bnbvalue = (_numberOfTokens/1000000000000000000)*tokenPrice;  
    require(msg.value >= _bnbvalue);  
    // ...  
    require(  
        tokenContract.transfer(msg.sender, _numberOfTokens), "Transfer failed");  
    // ...  
}
```

Underflow to 0 if {_numberOfTokens} too small

Pass this

Transfer arbitrary tokens

~~—Fixed price, vulnerable to price manipulation—~~

“Unconditional Token Gain” and +20% more profits by SGD



RQ2: Ablation

- 50% less targets without action based mutation
 - Much harder to satisfy constraints
- 6.7% profits without SGD exploitation
 - Random mutation is hard to exploit all profit potential
- 4.1% profits without profit accounting
 - Precise oracle is crucial



RQ3: Real-world Application

- Apply VERITE on 4 projects on Secure3
 - Need auditors to deploy contracts firstly
 - Fuzzing from empty states and before/after any active transactions
- 6 vulnerabilities found with PoP
 - 5 are high severity
- \$2500 bounty in total

Conclusion



VERITE: PoP Hunting Fuzzer

- Action based exploration + SGD based exploitation
- State-of-art profitable vulnerability fuzzer
- Even outperform on-chain attack transactions
- No false positive
- Designed for on-chain arm race

Q&A

Off the menu...



Auto Actions Extraction

- VERITE requires manual extraction
- *Hard* for custom business models
- We prototyped LLM-assisted actions extractions...
 - It works (roughly)
 - But still need better tuning

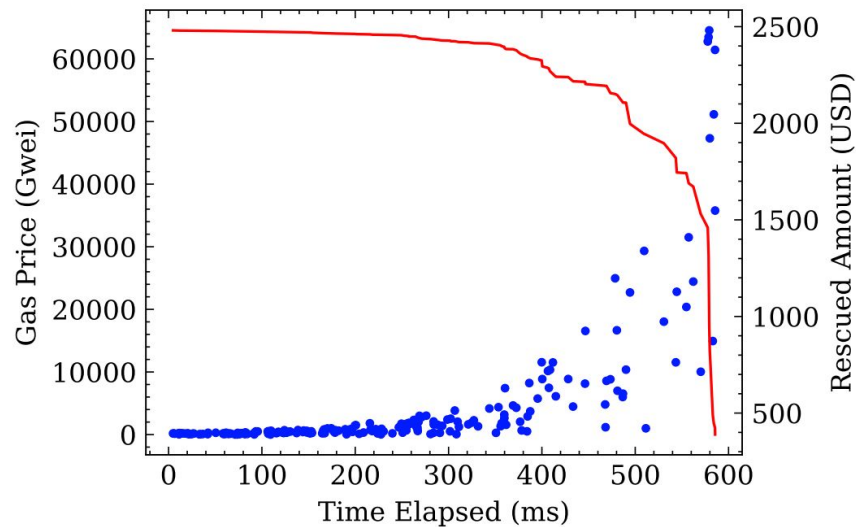
```
// Function to sell a percentage of base tokens for quote tokens
function sellBaseTokens(address to, uint256 percentage) external returns (uint256) {
    uint256 totalBase = baseToken.balanceOf(msg.sender);
    uint256 amountToSell = (totalBase * percentage) / 100;

    require(amountToSell > 0, "Amount must be greater than zero");
    require(baseToken.transferFrom(msg.sender, address(dodo), amountToSell), "Transfer failed");

    uint256 receivedAmount = dodo.sellBase(to);
    return receivedAmount;
}
```

Better Optimizer/Solver

- SGD is way too heavy and slow
- On-chain competition needs to solve optimal inputs in 1-2 seconds
 - e.g. frontrunning
- We prototyped a few other solvers working better than SGD after the submission
 - Solver speed is crucial in the arm race





Recent Incidents

- VERITE **can reproduce** some recent incidents not included in the dataset
 - Some with even higher profits, too
- We call for community to defense on-chain contracts
 - Some mistakes are obvious and repeated too many times
 - Audited != Secured != Unexploitable (Recall Bybit Safe Wallet incident)
- Personally, vastly biased takeaway: do not trust any audit report
 - *On-chain arm race participants are much more proficient than auditors!*



On-chain Security: Are we there yet?

- NO!
- On-chain security is way too underexplored
 - VERITE only cares about a small subset of profitable vulnerabilities
 - ... though these already lead to millions of profits
- What's else we can do?
 - *Fully automation*, like continuously from detection to defense
 - More software technique integration, like practical symbolic execution in 200ms
 - Engineering arm race is also an open research problem, like reverse engineering what other competitors have integrated

Q&A
